

ÁP DỤNG THUẬT TOÁN DFS, BFS TÌM ĐƯỜNG ĐI NGẮN NHẤT GIỮA HAI ĐỈNH CỦA ĐỒ THỊ

1. Một số bài toán áp dụng

Bài 1: Xe buýt.

Để phục vụ cho seagame 26, Indonesia đã chuẩn bị rất chu đáo về mọi mặt, đặc biệt là phương tiện giao thông cho các vận động viên. Indonesia đã xây dựng hệ thống xe buýt đi lại giữa một số địa điểm, các tuyến xe buýt đều là đường đi 2 chiều. Các hoạt động của seagame được tổ chức ở N địa điểm ($1 \leq N \leq 100$). Các địa điểm được đánh số từ 1 đến N. Các trạm dừng xe buýt đều được đặt ở các địa điểm tổ chức Seagame. Tất nhiên vì điều kiện không cho phép nên có một số địa điểm không có trạm xe buýt và giữa một số địa điểm không thể đi lại bằng xe buýt. Em hãy giúp các vận động viên cũng như khán giả xác định được từ địa điểm A đến địa điểm B nào đó có thể đi được bằng xe buýt không? Nếu đi được thì nên đi theo tuyến như thế nào để số lần dừng lại ở các trạm là ít nhất.

Dữ liệu vào: XEBUYT.INP

- Dòng đầu tiên ghi số N ($1 \leq N \leq 100$).
- Dòng thứ 2 ghi 2 số nguyên thể hiện địa điểm A và B.
- Các dòng tiếp theo, mỗi dòng ghi 2 số i và j cho biết tuyến xe buýt giữa 2 điểm i và j.

Kết quả: ghi ra file XEBUYT.OUT

Nếu không có đường đi thì ghi ra số -1, ngược lại ghi tuyến đường đi từ A đến B mà qua ít trạm nhất.

Ví dụ:

XEBUYT.INP	XEBUYT.OUT	XEBUYT.INP	XEBUYT.OUT
9	2->7->6->3	9	-1
2 3		2 8	
1 3		1 3	
1 6		1 6	
2 7		2 7	
3 6		3 6	
4 8		4 8	
6 7		6 7	
8 9		8 9	

Ý tưởng thuật toán:

- Gọi hàm BFS(A), sau khi kết thúc lời gọi hàm kiểm tra mảng đánh dấu tại đỉnh B, nếu $DD[B]=false$ thì đưa ra -1, ngược lại gọi hàm tìm đường đi $path(A,B)$

Chương trình tham khảo

```
#include <bits/stdc++.h>
using namespace std;
int n , m, A, B;
vector <int> ke[103];
int parent[103];
bool DD[103] ;
void nhap(){
    freopen("XEBUYT.INP","r",stdin);
    freopen("XEBUYT.OUT","w",stdin);
    cin >> n ;
    cin>>A>>B; cout<<n<<A<<B;
    for ( int i = 0 ; i< n ; i++ ) {
        int x , y ; cin >> x >> y ;
        ke[x].push_back(y) ;
        ke[y].push_back(x) ;
    }
    memset(DD , false , sizeof(DD)) ;
}
//=====
void BFS(int u) {
    queue <int> q ;
    q.push(u) ;
    DD[u]=true;
    while (!q.empty() ) {
        int v = q.front() ;
        q.pop() ;
        for (int x: ke[v]) {
            if ( !DD[x] ) {
                q.push(x);
                DD[x]=true;
                parent[x]=v;
                if(x==B) return;
            }
        }
    }
}
//=====
void path(int s, int t)
{
    BFS(s) ;
    if (!DD[t] ) cout<< -1 ;
    else {
        vector <int> res ; // dùng vector res để lưu đường đi
        while( t !=s ){
            res.push_back(t) ;
            t = parent[t] ;
        }
        res.push_back(s);
        reverse(res.begin(), res.end()) ;
        for(int i =0;i<res.size()-1;i++) cout<<res[i]<<"->";
        cout<<res.back();
    }
}
```

```

}
//=====
int main()
{
    nhap();
    path(A,B);
    return 0;
}

```

Bài 2 : Con đường

Một đất nước có N thành phố. Trên một mặt phẳng tọa độ, mỗi thành phố coi là một điểm. Tọa độ của các điểm này có giá trị tuyệt đối nhỏ hơn 30.000. Người ta muốn xây dựng các con đường thẳng từ thành phố này tới thành phố khác. Bản thiết kế xây dựng có các tính chất sau : Không có ba thành phố nào nằm trên một con đường, có không quá N-1 con đường, các con đường có thể giao nhau (do đó có thể chuyển sang con đường khác hoặc đi thẳng nếu gặp điểm giao). Bạn hãy lập trình kiểm tra xem từ thành phố 1 có thể đi đến được những thành phố nào ?

Dữ liệu vào từ file ROAD.INP có dạng :

- Dòng thứ nhất là 2 số nguyên N, M ($N \leq 200$), M là số con đường.
- N dòng sau, mỗi dòng 2 số nguyên mô tả tọa độ của một thành phố.
- M dòng tiếp theo mỗi dòng 2 số i, j mô tả có tuyến đường giữa 2 thành phố i và j.

Kết quả ghi ra file ROAD.OUT có dạng :

- Dòng đầu là số K : số thành phố tới được từ thành phố 1
- Dòng 2 là K số nguyên là các thành phố đến được (theo thứ tự từ nhỏ đến lớn).

Ví dụ :

Road.Inp	Road.out	Road.Inp	Road.out
4 2	4	4 2	2
0 0	1 2 3 4	0 0	1 2
1 0		1 0	
0 1		0 1	
1 1		1 1	
1 4		1 2	
2 3		3 4	

Ý tưởng thuật toán:

- Xây dựng ma trận kề: Dựa vào điều kiện cắt nhau của 2 cạnh (hai đoạn thẳng)

+ Do không có 3 thành phố nằm trên 1 con đường (không có 3 điểm thẳng hàng), nên 2 cạnh AB và CD cắt nhau (hai đoạn thẳng AB và CD cắt nhau) khi A, B khác phía so với đường thẳng CD và C,D khác phía so với đường thẳng AB.

Xây dựng hàm kiểm tra 2 cạnh AB, CD có cắt nhau hay không. Nếu CD cắt AB thì đẩy đỉnh A, B vào danh sách kề của C, của D; đẩy C, D vào danh sách kề của A và của B.

- Dùng *vector* **res** lưu lại các đỉnh được thăm đến trong hàm BFS hoặc DFS.
- Gọi hàm BFS(1) hoặc DFS(1) sau khi kết thúc lời gọi đưa kết quả của vector **res** ra.

Chương trình tham khảo

```
#include <bits/stdc++.h>
using namespace std;
const int N = 200;
struct point{
    int x, y;
};
point a[N+3];
int n, m, ans = 0;
vector<pair<int, int>> canh;
vector<int> ke[N+2];
int parent[N+3];
bool DD[N+3];
vector<int> res;
bool check(point A, point B, point C)
{
    int kt = (B.y - A.y) * (C.x - B.x) - (B.x - A.x) * (C.y - B.y);
    return kt > 0;
}
//kiểm tra đoạn thẳng AB có cắt CD không
bool cut(point A, point B, point C, point D){
    if(((check(A, B, C) && check(A, B, D)) == false) && ((check(C, D, A) &&
check(C, D, B)) == false)) return true;
    return false;
}
void nhap(){
    freopen("ROAD.INP", "r", stdin);
    freopen("ROAD.OUT", "w", stdout);
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
        cin >> a[i].x >> a[i].y;
    for(int i = 1; i <= m; i++)
    {
        int d1, d2;
        cin >> d1 >> d2;
        ke[d1].push_back(d2);
        ke[d2].push_back(d1);
        canh.push_back({d1, d2});
    }
}
```

```

        memset(DD,0,sizeof(DD));
    }
    void dske()
    {
        for(int i=0;i<m-1;i++)
        {
            for(int j=i+1;j<m;j++)

                if(cut(a[canh[i].first],a[canh[i].second],a[canh[j].first],a
[canh[j].second]))
                {
                    ke[canh[i].first].push_back(canh[j].first);
                    ke[canh[i].first].push_back(canh[j].second);
                    ke[canh[i].second].push_back(canh[j].first);
                    ke[canh[i].second].push_back(canh[j].second);
                    ke[canh[j].first].push_back(canh[i].first);
                    ke[canh[j].first].push_back(canh[i].second);
                    ke[canh[j].second].push_back(canh[i].first);
                    ke[canh[j].second].push_back(canh[i].second);
                }
        }
    }
    void DFS(int u)
    {
        DD[u]=true;ans++; res.push_back(u);
        for(int v:ke[u])
        {
            if(!DD[v])
            {
                DFS(v);
            }
        }
    }
    void xuly(){
        dske();
        DFS(1);
        cout<<ans<<endl;
        sort(res.begin(), res.end()) ;
        for(int i:res)
            cout<<i<<" ";

    }

    int main()
    {
        nhap();
        xuly();
        return 0;
    }

```

2. Các bài tập tự luyện

Bài 1: Mạng máy tính

Một mạng gồm N máy tính đánh số từ 1 đến N , và M kênh truyền tin một chiều giữa một số cặp máy trong mạng được đánh số từ 1 đến M . Mạng máy tính là thông suốt, nghĩa là từ một máy có thể truyền tin đến một máy bất kỳ khác trong mạng bằng các đường nối trực tiếp hoặc thông qua các máy trung gian. Một máy trong mạng là số chẵn nếu số kênh truyền tin trực tiếp từ nó đến các máy khác là chẵn. Một máy trong mạng là số lẻ nếu số kênh truyền tin trực tiếp từ nó đến các máy khác là lẻ. Giả sử s và t là hai máy lẻ trong mạng, hãy đổi hướng truyền tin của một số kênh để biến đổi mạng đã cho thành mạng (không nhất thiết phải thông suốt) mà trong nó hai máy s và t trở thành hai máy chẵn và không làm thay đổi tính chẵn lẻ của các máy khác trong mạng. Số kênh đổi hướng càng ít càng tốt.

Dữ liệu vào từ file văn bản NET.INP:

- Dòng đầu tiên chứa 2 số N, M ($N \leq 200, M \leq 1000$).
- Dòng thứ 2 chứa 2 số s và t .

NET.INP	NET.OUT
6 9	3
1 6	1
1 2	7
2 3	9
3 4	
4 1	
4 6	
6 3	
2 5	
5 3	
5 6	

1 hệ
cho:
trên

Bài 2: Khu vui chơi

Để lôi kéo khách du lịch nước ngoài, Việt Nam đã xây dựng thống N điểm vui chơi, đánh số từ 1 đến N . Các điểm vui chơi được nối bằng các đoạn đường trực tiếp, hai chiều, đảm bảo sao Giữa 2 địa điểm bất kỳ có đúng 1 đường đi (trực tiếp hay gián tiếp). Độ dài của đường đi này bằng số các đoạn đường trực tiếp đường đi đó.

Ban quản lý khu vui chơi có 1 hành trình bắt đầu từ địa điểm A_1 lần lượt qua các địa điểm $A_2, A_3, \dots, A_m$. Vì các du khách muốn biết trước được tổng chiều dài hành trình mà họ phải đi qua nên ban quản lý quyết định thuê bạn để tính độ dài này. Biết rằng độ dài hành trình bằng tổng độ dài các đường đi trên hành trình đó.

Dữ liệu vào: Được ghi trong file văn bản VC.INP có cấu trúc:

- Dòng đầu tiên chứa 2 số nguyên dương N và M với $N \leq 250; M \leq N$.
- $N-1$ dòng tiếp theo, mỗi dòng ghi 2 số nguyên dương u, v mô tả có con đường nối 2 địa điểm u và v .
- Dòng cuối cùng gồm M số lần lượt là M địa điểm trên hành trình

Dữ liệu ra: Kết quả ghi vào file văn bản **VC.OUT** chỉ có 1 số là tổng độ dài hành trình

Ví dụ:

VC.INP	VC.OUT
8 5	15
1 4	
1 2	
3 2	
3 8	
1 5	
3 6	
2 7	
4 8 5 6 7	

Bài 3: Con khỉ đi tìm chuối.

Một con khỉ đói muốn ăn chuối. Khỉ và này chuối ở trong một mê cung gồm các phòng và hành lang nối các phòng. Mỗi phòng chỉ có một trong hai trạng thái: khóa hoặc không khóa cửa. Nếu phòng bị khóa thì khỉ không thể vào phòng này, nhưng có thể trèo ra khỏi nó. Phòng không khóa thì khỉ có thể vào ra tự do. Một số phòng có công tắc, ấn công tắc sẽ thay đổi trạng thái của một số phòng, nghĩa là những phòng này đang khóa thành không bị khóa và ngược lại. Trên lối vào phòng có công tắc, khỉ có thể ấn công tắc nếu nó muốn. Viết chương trình giúp đỡ khỉ tìm được phòng có chuối càng sớm càng tốt, nghĩa là tìm số ít nhất các hành lang khỉ phải qua để tìm đến phòng có chuối, có thể ấn một vài công tắc. Dữ liệu vào miêu tả các phòng và hành lang trong mê cung, trạng thái ban đầu của các phòng, danh sách các công tắc và với mỗi công tắc có một danh sách các phòng bị thay đổi trạng thái bởi công tắc đó.

Dữ liệu vào từ file văn bản MONKEY.INP:

- Dòng đầu tiên gồm 2 số nguyên N và S. Số N là tổng số phòng ($1 \leq N \leq 100$) và số S là số công tắc, cũng là số phòng có công tắc trong mê cung ($1 \leq S \leq 8$). Các công tắc được đặt trong các phòng được đánh số từ 1 đến S.

- N dòng tiếp theo mô tả các phòng : phòng thứ i được mô tả trên dòng i+1, bắt đầu bằng số 0 nếu phòng này ban đầu không khóa, bằng 1 nếu bị khóa, tiếp theo là số K là số phòng có hành lang nối tới phòng thứ i, sau đó là K số thể hiện số hiệu các phòng này. Các số trên một dòng cách ít nhất một kí tự trống.

- S dòng tiếp theo mô tả S công tắc (nằm từ phòng thứ đến phòng thứ S): mỗi dòng được bắt đầu bằng số nguyên L, là số lượng phòng bị thay đổi trạng thái bởi công tắc đang mô tả. Tiếp theo là L số, là các số hiệu các phòng bị tác động thay đổi trạng thái do công tắc này.

- Dòng cuối cùng là 2 số A và B, A là số hiệu phòng khi đang đứng, B là số hiệu phòng có chuối.

Kết quả ghi ra file văn bản MONKEY.OUT: chỉ gồm một dòng ghi duy nhất một số là số các hành lang mà khỉ qua để đến phòng có chuối.

Ví dụ:

MONKEY.INP	MONKEY.INP	MONKEY.INP
4 1	5 2	6 2
0 1 3	0 2 2 5	0 2 6 5
1 2 3 4	1 2 1 3	1 2 4 6
0 2 1 2	0 2 2 4	0 1 4
0 1 2	1 2 3 5	1 3 2 5 3
1 2	0 2 1 4	0 3 1 4 6
3 4	2 2 4	0 3 1 5 2
MONKEY.OUT	2 3 4	3 2 5 3
4	5 3	1 4
	MONKEY.OUT	6 3
	3	MONKEY.OUT
		3

Bài 4:

<https://oj.vnoi.info/problem/c11bc2>